create an Architectural Decision Records (ADR) document, which evidences the technical criteria, restrictions and justifications of the selected technologies, including the sources.

ADR documents should document the decision context, selected technology, and consequences, evidence technical criteria including performance and quality attributes, specify restrictions such as time and cost constraints, justify selections through trade-off and cost-benefit analyses aligned with business goals, and cite supporting evidence from empirical studies, expert opinions, and standards using Markdown-based or structured templates.

Abstract

Architectural decision records for technology selection should document core elements including rationale or context, the decision itself, and consequences, with more detailed templates specifying problem, motivation, cause, solutions, choice, and architectural modification. Technical criteria to evidence include performance metrics such as response time requirements and scalability, quality attributes like reliability, security, and maintainability, integration capabilities, and cost considerations. Restrictions affecting technology selection encompass time constraints that lead to incomplete documentation, organizational constraints including company size and development methodology, cost limitations, and knowledge vaporization that increases maintenance costs. Justifications should address trade-offs between ease of implementation versus performance, cost-benefit analyses where significantly higher costs influenced decisions, and alignment with business goals such as reduced cost and long-term scalability.

Evidence sources cited to support architectural decisions include empirical studies and benchmarks such as evaluations on large open-source systems, expert opinions including tacit knowledge of architects, previous project experiences, standards and best practices like ISO/IEC/IEEE 42010, and requirements documents. Documentation formats range from Markdown-based templates to structured frameworks with multiple viewpoints, with Michael Nygrad's template widely adopted. However, practical adoption faces challenges, with 50% of repositories containing only 1-5 ADRs, suggesting that while comprehensive templates and frameworks exist for documenting technical criteria, restrictions, and justifications, sustained use requires integration into developer workflows and team-based recording activities to overcome time pressure and productivity concerns.

Paper search

We performed a semantic search across over 138 million academic papers from the Elicit search engine, which includes all of Semantic Scholar and OpenAlex.

We ran this query: ”create an Architectural Decision Records (ADR) document, which evidences the technical criteria, restrictions and justifications of the selected technologies, including the sources.”

The search returned 50 total results from Elicit.

We retrieved 50 papers most relevant to the query for screening.

Screening

We screened in sources based on their abstracts that met these criteria:

- **ADR Focus**: Does this study focus on Architectural Decision Records (ADRs) or architectural decision documentation?

- **ADR Methods**: Does this research address methods, processes, or frameworks for creating ADRs?
- **Technical Decision Criteria**: Does this study examine technical criteria, restrictions, or justifications in technology selection decisions?
- **Relevant Context**: Was this research conducted in software development, system architecture, or IT project contexts?
- **Study Type**: Is this an empirical study, case study, experience report, systematic review, or meta-analysis?
- **Architectural Decision Specificity**: Does this study specifically reference architectural decisions rather than focusing solely on general software documentation?
- **Decision Rationale**: Does this research address decision-making processes or justification aspects rather than being limited to high-level architectural patterns without decision rationale?
- **Evidence-Based Content**: Does this study provide empirical evidence or practical application rather than being purely theoretical, opinion-based, or editorial content?

We considered all screening questions together and made a holistic judgement about whether to screen in each paper.

Data extraction

We asked a large language model to extract each data column below from each paper. We gave the model the extraction instructions shown below for each column.

- **Technical Criteria**:

  Extract all technical criteria mentioned for evaluating or selecting architectural decisions and technologies, including:

- Performance criteria (speed, throughput, latency, scalability)
- Quality attributes (reliability, security, maintainability, usability)
- Technical compatibility requirements
- Integration capabilities
- Resource requirements (memory, CPU, storage)
- Development and operational complexity
- Technology maturity and stability Focus specifically on criteria that could be applied to technology selection decisions.

- **Constraints and Restrictions**:

  Extract all constraints, restrictions, and limitations identified that affect architectural decisions, including:

- Budget/cost limitations
- Time constraints and deadlines
- Organizational policies and standards
- Regulatory and compliance requirements
- Existing technology stack limitations
- Team skill sets and expertise
- Infrastructure constraints
- Security and governance restrictions
- Vendor lock-in concerns Capture both hard constraints (must/cannot) and soft constraints (preferences).

- **Decision Justifications**:

  Extract the rationale and justifications provided for architectural and technology decisions, including:

- Why specific technologies were chosen over alternatives
- Trade-offs that were considered and accepted
- Risk assessments and mitigation strategies
- Business value and alignment arguments
- Technical advantages cited
- Cost-benefit analyses mentioned
- Future-proofing considerations Focus on explicit reasoning rather than just the decisions themselves.

- **Evidence Sources**:

  Extract all types of evidence and sources cited to support architectural decisions, including:

- Empirical studies and benchmarks
- Industry reports and whitepapers
- Vendor documentation and specifications
- Community feedback and reviews
- Previous project experiences
- Expert opinions and consultations
- Proof-of-concept results
- Market analysis and trends
- Standards and best practices references Note the credibility and type of each source mentioned.

- **ADR Content Elements**:

  Extract recommended content elements and sections for Architectural Decision Records, including:

- Required fields or sections (title, status, context, decision, consequences)
- Optional information elements
- Metadata requirements (dates, stakeholders, approval)
- Template structures and formats
- Documentation standards mentioned
- Traceability requirements
- Review and update processes Focus on what information should be captured in ADR documents.

- **Decision Context**:

  Extract contextual information that influences architectural decisions, including:

- Project phase or stage when decisions are made
- Stakeholders involved in decision-making
- Organizational context and culture
- System requirements and constraints
- Technical debt considerations
- External factors (market, competition, regulations)
- Timeline and urgency factors
- Resource availability Capture context that affects how decisions should be documented and justified.

- **Tools and Methods**:

  Extract tools, methods, and approaches mentioned for capturing, documenting, and managing architectural decisions, including:

- Software tools and platforms
- Documentation formats (Markdown, templates, frameworks)
- Decision-making frameworks and methodologies
- Knowledge management approaches
- Integration with development workflows
- Collaboration and review processes
- Version control and change management Focus on practical approaches for creating and maintaining ADRs.

- **Quality Evaluation**:

  Extract criteria and methods for evaluating the quality and effectiveness of architectural decision documentation, including:

- Completeness metrics
- Clarity and understandability measures
- Traceability and linkage quality
- Maintenance and update frequency
- Stakeholder satisfaction indicators
- Usage and adoption metrics
- Decision outcome tracking
- Knowledge preservation effectiveness Capture how to assess whether ADRs are serving their intended purpose.

Results

Characteristics of included studies

The review included 10 studies examining architectural decision documentation practices, tools, and frameworks. Three studies provided full text access, while seven were available as abstracts only.

Study	Full text retrieved?	Study focus	Year	Primary contribution
Arman Shahbazian et al.	Yes	Automated recovery of architectural design decisions	2018	RecovAr technique for recovering decisions from project history with 75% recall and 77% precision
U. V. Heesch et al.	No	Documentation framework for architecture decisions	2012	Four viewpoints based on ISO/IEC/IEEE 42010 standard
Aleksei Karetnikov et al.	No	Semantic modeling of ADRs using knowledge graphs	2024	AI-based analysis approach for over 4,000 open-source ADRs from GitHub

Study	Full text retrieved?	Study focus	Year	Primary contribution
Georg Buchgeher et al.	No	Mining software repositories study of ADR adoption	2023	Analysis showing low but increasing ADR adoption, with 50% of repositories containing only 1-5 ADRs
Oliver Kopp et al.	No	Markdown-based ADR template and tool support	2018	Integration of decision capturing into developer toolchain during development iterations
D. Tofan et al.	No	Web-based tool for capturing and analyzing architectural decisions	2014	Open source tool supporting group decision making based on theoretical foundations
Diego Dermeval et al.	No	STREAM-ADD process extension	2012	Systematic documentation of architectural decisions during requirements-to-architecture transition
R. Capilla et al.	No	Architecture Design Decision Support System (ADDSS)	n.d.	Web-based tool for maintaining trace links between decisions and software artifacts
A. Jansen et al.	Yes	ADDRA approach for recovering architectural decisions after the fact	2008	Five-step iterative process using architectural deltas to recover decisions
Efi Papatheocharous et al.	Yes	GRADE taxonomy for decision support	2015	Framework with goals, roles, assets, decision methods, criteria, and environment elements

The studies represented diverse approaches to architectural decision documentation, including automated recovery techniques, manual documentation frameworks, web-based tools, and semantic analysis methods. Recording deci-

sions as a team activity over extended periods was observed in repositories that systematically used ADRs. Most approaches focused on capturing decisions during development phases, though some specifically addressed post-hoc recovery when documentation was missing or incomplete.

Thematic analysis

Technical criteria for architectural decisions

Technical criteria mentioned across studies fell into several categories. Performance criteria included response time requirements, such as submissions being tested and reported within 10 seconds, as well as scalability considerations. Quality attributes emphasized reliability, security, maintainability, and ease of implementation. The assessment of architectural decisions considered rationale clarity and cohesion, consequence-rationale association, and concerns with higher-level architectural quality.

Integration capabilities were noted as technical selection factors, with distributed database syncing mentioned as a capability not available in all options. Cost emerged as a resource requirement consideration, alongside development and operational complexity evaluated through ease of implementation. However, most abstract-only studies did not provide specific technical criteria details.

Constraints and restrictions affecting architectural decisions

Multiple constraint types influenced architectural decision-making. Documentation-related constraints included lack of documentation, incomplete or unreliable documentation, and limited access to architects who may not remember or articulate past decisions. Knowledge vaporization increased maintenance costs and exacerbated architectural drift and erosion.

Time constraints significantly affected documentation practices, with time pressure leading to incomplete or missing documentation. Practices perceived as time-wasters or causing context switches and media breaks that harm productivity were actively avoided. Organizational constraints included company size of 400 people, development unit size of 32 people, and use of hybrid development methodology combining agile and plan-driven approaches.

Cost limitations affected technology selection, with one case showing that the Microsoft option cost significantly more than MySQL. Methodological constraints included vagueness about what constitutes an architectural design decision, availability and accuracy of artifacts affecting recovery process reliability, assumptions about architect involvement limiting task delegation, subjectivity of architectural views affecting comparability and objectivity, and inability to always recover considered alternatives and trade-offs.

Decision justifications and rationale

Justifications for architectural decisions centered on trade-offs and technical advantages. Trade-offs between ease of implementation versus performance were explicitly considered, with preferences sometimes favoring ease of implementation. Trade-offs between costs and performance represented another dimension of decision-making.

Technology selection justifications included lower cost alignment with business goals, reduced cost and long-term scalability as goal-prerequisites, and alignment with business objectives. Technical advantages cited the use of application generic knowledge and patterns, implying future-proofing considerations. Cost-benefit analyses influenced decisions, with significantly higher costs for some alternatives affecting final choices.

Accepted trade-offs included situations where some desired features were unavailable in the selected technology but could be addressed through other means. Decision criteria considered financial and product-related factors,

focusing on performance, reliability, security, cost, and scalability. However, most abstract-only studies did not provide detailed justification examples.

Evidence sources supporting architectural decisions

Studies employed diverse evidence sources to support architectural decisions. Empirical studies and benchmarks included evaluation on Hadoop and Struts using state-of-the-art architectural recovery techniques, industrial case studies, and mining software repositories studies analyzing GitHub repositories. Open-source ADRs from GitHub numbered over 4,000 pre-filtered records.

Expert opinions and consultations featured prominently, with tacit knowledge of original architects heavily relied upon, independent scoring by PhD students, use of Michael Nygrad's template, and contributions from ten international experienced researchers. Previous project experiences included reference to previous work and comparative analyses, templates derived from previous work, theoretical and conceptual foundations from previous work, and real industrial cases from consultancy activities.

Standards and best practices references encompassed ISO/IEC/IEEE 42010, specialization of generalized architecting models, and theories and standards used in categorizing elements. Requirements documents and specifications informed decision-making through requirements engineering processes. Proof-of-concept results included early validation in open source projects. Community feedback was reflected in analysis of open source repositories and tool availability in public GitHub repositories. Future validation through interviews with practitioners was planned.

ADR content elements and structure

Required fields and sections for ADRs showed consistency across studies. Core elements included rationale or context, the decision itself, and consequences. The most detailed template structure specified problem, motivation, cause, solutions, choice, and architectural modification as required components.

Optional information elements included rationale clarity, rationale cohesion, consequence-rationale association, consequence tractability, goals, roles, and assets. Documentation formats employed condensed style with bold elements followed by textual descriptions, graphical notation for architectural modifications, and asterisks for rationalized solutions.

Framework structures included four viewpoints: Decision Detail viewpoint, Decision Relationship viewpoint, Decision Chronology viewpoint, and Decision Stakeholder Involvement viewpoint, all based on ISO/IEC/IEEE 42010 conventions. Traceability requirements emphasized understanding rationale and consequences and maintaining trace links between design decisions and other software artifacts. Review and update processes focused on continuous capture and understanding of architectural decisions.

Decision context factors

Contextual factors influencing architectural decisions spanned multiple dimensions. Project phases included various stages during system evolution and maintenance, early software development during requirements engineering and architectural design, development iterations, and implementation phase when architecture is being realized.

Stakeholders involved architects and developers, coding architects and developers, architects and other stakeholders, architects and team members with tacit knowledge, and multiple team members in repositories with systematic ADR use. Their availability and ability to articulate decisions remained limited.

Organizational context reflected cultures prioritizing immediate delivery over documentation, lack of reliable documentation, focus on productivity and efficiency, company size of 400 people with development units of 32 people, and use of standardized templates. Development methodologies included hybrid approaches combining agile and plan-driven methods.

System requirements influenced decisions through requirements engineering processes and specifications for high-performance systems like HRM systems with choices between MySQL and Microsoft. Technical debt considerations centered on knowledge vaporization and challenges in maintaining documentation and recovering decisions. Timeline and urgency factors manifested as significant time pressure leading to quick decision-making and needs to reverse knowledge vaporization. Resource availability was limited, relying on tacit knowledge for decision recovery and requiring efficient resource utilization through automated techniques.

Tools and methods for ADR management

Software tools and platforms included RecovAr and UnArch for automated decision recovery, ARCADE as a software workbench providing architecture recovery techniques, GitHub as a platform for managing ADRs, web-based open source tools supporting group decision making, and ADDSS (Architecture Design Decision Support System). Recovery tools for detailed design and architectural views facilitated post-hoc documentation.

Documentation formats emphasized Markdown-based decision capturing templates and templates based on conceptual models for documenting decisions. Michael Nygrad's template was widely used in repositories. Structured frameworks included four viewpoints (Decision Detail, Decision Relationship, Decision Chronology, and Decision Stakeholder Involvement) based on ISO/IEC/IEEE 42010 conventions.

Decision-making frameworks included STREAM-ADD for transitioning from requirements to architectural models with decision documentation, ADDRA approach for recovering decisions after the fact, and GRADE taxonomy for structuring decisions. Knowledge management approaches employed knowledge graphs for analyzing ADRs and used tacit knowledge of original architects.

Integration with development workflows involved using issue trackers and version control repositories, derivation of ADRs from GitHub, integration into developer toolchains, and iterative processes involving recovery of detailed design and architectural views. Collaboration and review processes included recording decisions as team activities conducted by two or more users over longer periods and support for group decision making. Version control and change management utilized version control repositories, public GitHub repositories, and architectural deltas to track changes between releases.

Quality evaluation of ADRs

Completeness metrics demonstrated recall of 75% and precision of 77% for automated recovery techniques, with recall after eliminating externally caused changes reaching 73%. ADRs were designed to capture information about software architecture, decisions, context, and consequences. Repository analysis showed that 50% of repositories with ADRs contained only 1-5 records, suggesting limited completeness in many cases.

Clarity and understandability measures included rationale clarity and rationale cohesion, AI-based methods facilitating efficient identification of relevant knowledge, recording as team activities, and use of standardized templates like Michael Nygrad's. Benefits of systematic approaches included recovery of rationale.

Traceability and linkage quality was evaluated through consequence-rationale association and consequence tractability, modeling ontology and mapping to Wikidata concepts, recording as team activities, maintaining trace links between design decisions and other software artifacts using ADDSS, and highlighting traceability as a benefit of

recovering architectural design decisions. The taxonomy's ability to evaluate decisions retrospectively supported traceability assessment.

Maintenance and update frequency was reflected in creation of semantic models and derivation of over 4,000 ADRs from GitHub, with implications for preservation through documentation. Usage and adoption metrics showed low but increasing adoption rates, with repositories containing 1-5 ADRs representing 50% of those using ADRs. Decision outcome tracking was supported by taxonomy instantiation through real cases. Knowledge preservation effectiveness was measured by recall after eliminating externally caused changes, with implications through taxonomy use and recovery of architectural design decisions and rationale.

Synthesis

The studies revealed two distinct approaches to architectural decision documentation that may appear contradictory but actually address different organizational contexts and constraints. Three studies focused on automated post-hoc recovery of decisions, while seven emphasized prospective manual documentation frameworks and tools. This divergence reflects a fundamental tension in software development: time pressure often prevents adequate documentation during development, yet missing documentation increases maintenance costs and knowledge loss.

The automated recovery approaches achieved modest but useful results, with RecovAr demonstrating 75% recall and 77% precision, while manual documentation showed concerning adoption patterns—50% of GitHub repositories with ADRs contained only 1-5 records, suggesting that teams often tried but abandoned the practice. This pattern suggests that neither approach alone fully addresses the problem: automated recovery cannot capture all decision rationale and alternatives considered, while manual documentation struggles with perceived time costs and productivity impacts.

The divergent findings likely reflect different organizational maturity levels and project phases. Organizations lacking documentation face the recovery problem addressed by automated techniques, operating in what might be termed ”documentation debt” contexts where knowledge has already vaporized. In contrast, organizations implementing prospective frameworks operate in ”documentation prevention” contexts, attempting to avoid future recovery needs. The low completion rates in the GitHub study suggest that many organizations begin with prevention intentions but revert to debt accumulation under time pressure.

Context-sensitive recommendations emerge from this synthesis. For projects in early phases with available architects, lightweight templates integrated into developer workflows minimize context switching while capturing decisions when rationale is fresh. Recording as team activities over time appears more sustainable than individual documentation burdens. For projects with existing documentation gaps, automated recovery techniques combined with architect validation can efficiently address knowledge loss, accepting 73-75% recall as preferable to total knowledge vaporization. For long-term knowledge management, semantic approaches using knowledge graphs and structured taxonomies enable cross-project learning and analysis at scale.

The heterogeneity in evidence sources and evaluation criteria across studies limits direct comparability. Studies with full text access provided specific technical criteria like 10-second response time requirements and financial constraints, while abstract-only studies rarely specified such details. This suggests publication bias toward methodological contributions over practical implementation details. Future work should emphasize reporting specific decision criteria, actual constraints encountered, and quantitative adoption metrics to enable evidence-based practice recommendations.

References

A. Jansen, J. Bosch, and P. Avgeriou. "Documenting After the Fact: Recovering Architectural Design Decisions." *Journal of Systems and Software*, 2008.

Aleksei Karetnikov, Lisa Ehrlinger, Georg Buchgeher, and Verena Geist. "Semantic Modeling of Architecture Decision Records to Enable AI-Based Analysis." *IEEE International Conference on Software Analysis, Evolution, and Reengineering*, 2024.

Arman Shahbazian, Youn Kyu Lee, D. Le, Yuriy Brun, and N. Medvidović. "Recovering Architectural Design Decisions." *International Conference on Software Architecture*, 2018.

D. Tofan, and M. Galster. "Capturing and Making Architectural Decisions: An Open Source Online Tool." *ECSAW '14*, 2014.

Diego Dermeval, J. Pimentel, Carla Schuenemann, J. Castro, E. Santos, Gabriela Guedes, Márcia Lucena, and A. Finkelstein. "STREAM-ADD - Supporting the Documentation of Architectural Design Decisions in an Architecture Derivation Process." *2012 IEEE 36th Annual Computer Software and Applications Conference*, 2012.

Efi Papatheocharous, K. Petersen, A. Cicchetti, Séverine Sentilles, S. M. A. Shah, and T. Gorschek. "Decision Support for Choosing Architectural Assets in the Development of Software-Intensive Systems: The GRADE Taxonomy." *ECSA Workshops*, 2015.

Georg Buchgeher, Stefanie Schöberl, V. Geist, B. Dorninger, Philipp Haindl, and R. Weinreich. "Using Architecture Decision Records in Open Source Projects—An MSR Study on GitHub." *IEEE Access*, 2023.

Oliver Kopp, A. Armbruster, and O. Zimmermann. "Markdown Architectural Decision Records: Format and Tool Support." *Central-European Workshop on Services and Their Composition*, 2018.

R. Capilla, Juan C. Dueñas, F. Nava, R. Capilla, J. C. Due˜nas, and F. Nava. "Journal of Software Maintenance and Evolution: Research and Practice Viability for Codifying and Documenting Architectural Design Decisions with Tool Support," n.d.

U. V. Heesch, P. Avgeriou, and R. Hilliard. "The Journal of Systems and Software," 2012.